Chapter 10: Object-Oriented Programming

ENG3091

Khiem Nguyen

Contents

1 Why object-oriented programming? 2
- 1.1 Procedural versus object-oriented programming . . . 2
- 1.2 OOP is everywhere . . . 2
- 1.3 C++ is in high demand . . . 2
- 1.4 Software architecture at Airbus . . . 4
- 1.5 Git and OOP are essential skills . . . 4
- 1.6 The four pillars of OOP . . . 5
- 1.7 Encapsulation . . . 5
- 1.8 Encapsulation in C++: `public` and `private` . . . 6
- 1.9 Abstraction . . . 6
- 1.10 Abstraction: an example from graphics . . . 6
- 1.11 Encapsulation versus abstraction . . . 7
- 1.12 Inheritance . . . 7
- 1.13 Polymorphism . . . 7

2 Classes and objects 7
- 2.1 Classes and objects . . . 7
- 2.2 Defining a class . . . 8
- 2.3 Naming member variables . . . 8
- 2.4 `struct` versus `class` . . . 9
- 2.5 `struct` versus `class`: Google's C++ style . . . 10
- 2.6 A class can behave like a struct . . . 10
- 2.7 You've been using classes all along . . . 11

3 Member functions 11
- 3.1 A non-member function . . . 11
- 3.2 A member function . . . 12
- 3.3 The implicit object . . . 12
- 3.4 Coming from Python: where is `self`? . . . 13
- 3.5 Python checks late… . . . 13
- 3.6 …or never . . . 14
- 3.7 Member functions can be overloaded . . . 14
- 3.8 Structs and member functions: a bit of history . . . 15

4 `const` objects and `const` member functions 15
- 4.1 `const` objects . . . 15
- 4.2 `const` objects can't call non-`const` member functions . . . 15
- 4.3 `const` member functions . . . 16
- 4.4 A `const` member function can't change members . . . 17
- 4.5 `const` member functions work on any object . . . 17
- 4.6 Wait a minute: what a mess! . . . 18

4.7 `const` objects via pass by `const` reference . . . 18
4.8 Spot the problem . . . 18
4.9 The fix is one word . . . 19

5 Wrapping up **19**
5.1 Summary . . . 19
5.2 Keep learning . . . 19

1 Why object-oriented programming?

1.1 Procedural versus object-oriented programming

- So far we've written **procedural** programs: the data and the functions that work on it are separate things.
- In **object-oriented programming** (OOP), we create our own data types that bundle **data** (properties) together with **behaviour** (functions).

I won't spend long on *why* OOP matters:

- you met it in Introductory Programming 2 and other programming courses;
- since the 1990s, OOP has been the industry standard.

Instead, let me walk you through some real job adverts.

1.2 OOP is everywhere

- **It's natural:** it mirrors how we think about the world, as objects with properties and behaviours.
- **It scales:** it makes large, complex systems manageable, by *encapsulating* data so bugs can't spread.
- **It's what industry uses:** look at any job advert or large codebase.

Responsibilities

After training on MathWorks development processes and tools, you will work with a development team on implementing features such as algorithms or UIs for a future release of one of our 100+ products, or help to develop automated tests to verify that features in MATLAB or Simulink are working as designed.

This internship will prepare you for a career at MathWorks starting in our Engineering Development Group (EDG).

Qualifications

- Pursuing a MS or PhD in Computer Science, Engineering, or another STEM subject, and be enrolled in education at the time of the internship
- Experience with object orientated design and in programming languages such as: C++, JavaScript, Python, Go or MATLAB
- Passion for learning new technologies

You don't need MATLAB to get a job at MathWorks. You need C++ and OOP.

1.3 C++ is in high demand

I asked Gemini about software engineering roles at MathWorks, the company that makes the MATLAB you use all the time:

It is **very right** to say that for most core software engineering roles at MathWorks, C++ is a primary requirement.

Because MathWorks builds high-performance tools like **MATLAB** and **Simulink**, they need a language that can handle heavy numerical computation and memory management—which is exactly what C++ is designed for.

Here is a breakdown of why C++ is so critical there and what else they look for:

---

1. Why MathWorks depends on C++

Most of the "under the hood" engine of MATLAB and Simulink is written in **C++** and **C**.

- **Performance:** High-speed matrix math and simulation require the low-level control that C++ provides.
- **Code Generation:** Many MathWorks products (like MATLAB Coder) are specifically designed to turn MATLAB code *into* C++ code for use in cars, planes, and medical devices.
- **Legacy & Scale:** With millions of lines of code, their infrastructure relies on the robust Object-Oriented patterns found in modern C++.

---

2. When is C++ *not* required?

While C++ is the "king" at MathWorks, it isn't the only way in. Depending on the team, they look

1.4 Software architecture at Airbus

ABOUT YOU

- Bachelor's or Master's degree in Computer Science, Software Engineering, or a related technical field.
- 8-10 years of progressive experience in software development, with at least 3-5 years in a dedicated Software Architect role.
- Proven experience in designing and delivering complex, large-scale software systems, preferably within the defence, aerospace, or other highly regulated industries.
- Demonstrable experience with various architectural patterns (e.g., microservices, event-driven, monolithic decomposition) and their appropriate application.
- Strong proficiency in multiple programming languages (e.g., C++, Java, Python, Ada) relevant to embedded or real-time systems.
- Extensive knowledge of software design principles, object-oriented design, and design patterns.
- Experience with real-time operating systems (RTOS) and embedded software development is highly desirable.
- Familiarity with cloud computing platforms (e.g., AWS, Azure, Google Cloud) and containerization technologies (e.g., Docker, Kubernetes) where applicable.
- Understanding of cybersecurity principles and secure software development practices.
- Experience with model-driven architecture (MDA) tools and techniques is a plus.

Not a 100% match? No worries! Airbus supports your personal growth with customised development solutions.

1.5 Git and OOP are essential skills

Note the requirement for Java experience. For the record, Java is an object-oriented language through and through.

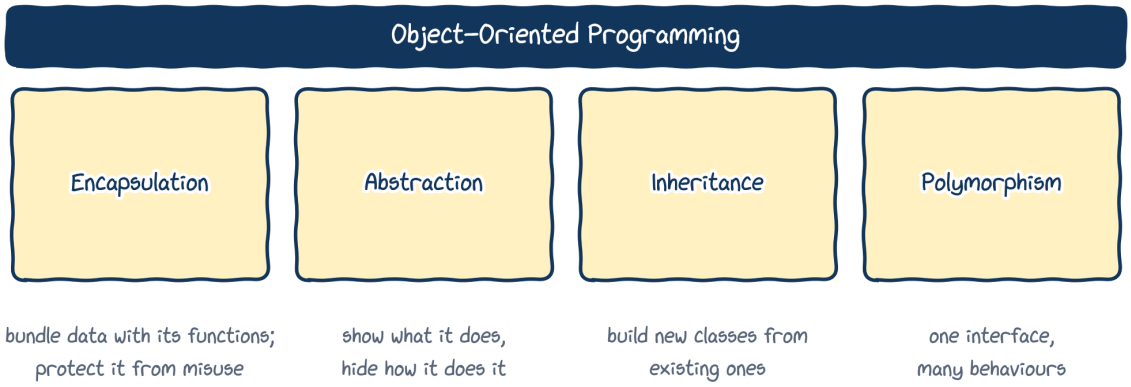
ABOUT YOU

- Bachelor's or Master's degree in Computer Science, Software Engineering, or a related technical field.
- Proven experience in software development, preferably within a professional environment.
- Strong experience with Java (17+),
- Experience with software development methodologies (e.g., Agile, Scrum).
- Familiarity with version control systems (e.g., Git).
- Understanding of software testing principles and experience with unit and integration testing.

Not a 100% match? No worries! Airbus supports your personal growth with customised development solutions.

1.6 The four pillars of OOP

I hope that's more than enough to convince you. But it's a lazy explanation!



Let's go through what each pillar is, and why it's there.

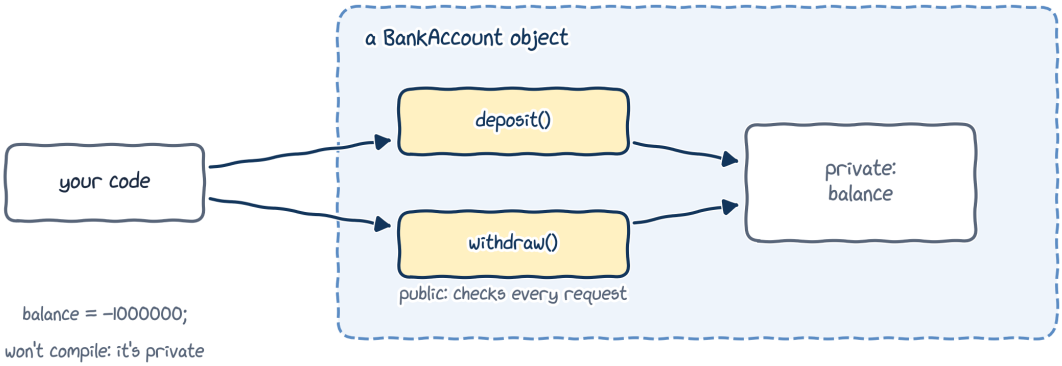
1.7 Encapsulation

Encapsulation means bundling data and the functions that work on it into one unit, a **class**, and hiding the data from the outside world, to prevent accidental interference.

You've already relied on it in Python:

- A NumPy `ndarray` or a Matplotlib `Figure` hides its internals; you just use it.
- `np.array([1, 'two', 3])` with `dtype=float` fails with a clear message about exactly what's wrong: plenty of checking happens behind the scenes, and you never need to see it.
- Registering a student as `Student(name="", age=-7)` should fail. Some code, somewhere, has to check the data and tell the user.

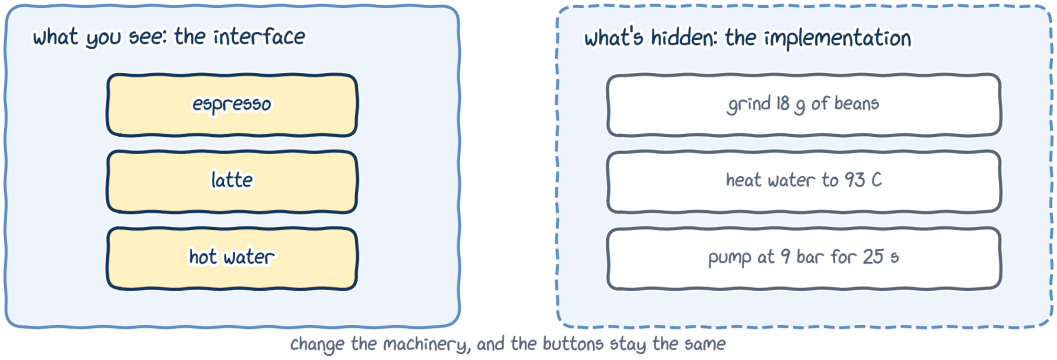
1.8 Encapsulation in C++: `public` and `private`



C++ uses **access specifiers**: `private` members can only be used inside the class; `public` members can be used from outside. If anyone could change a bank account's `balance` directly, the bank would be in serious trouble. Instead, you go through `deposit()` and `withdraw()`, which check that each transaction is valid.

1.9 Abstraction

Abstraction means showing only what an object **does**, and hiding **how** it does it.



- **Less complexity:** users deal with a simple interface.
- **Easier maintenance:** developers can change the hidden machinery without changing how anyone uses it.

1.10 Abstraction: an example from graphics

We want to draw different shapes, but a circle and a square are drawn differently. So we write an **abstract class** `Shape` that says "anything that is a shape must have a `draw()` function":

```
class Shape {
public:
    virtual void draw() = 0; // "= 0": every real shape must provide draw()
};
```

Code that draws shapes only needs to know about `draw()`, not about circles and squares. We'll write classes like this in Chapter 15.

1.11 Encapsulation versus abstraction

- **Encapsulation** is about **protection and data integrity**: refusing a negative `battery_level`, or a `withdrawal` larger than the balance.
- **Abstraction** is about **simplification and interfaces**: the user just calls `set_battery(50)`, without ever seeing the `if (value < 0)` check inside.

My own opinion:

- I've tried hard to find examples that separate the two cleanly. Sometimes I get confused myself!
- Many examples sit at the intersection of both.
- Don't worry if you're a little confused after this slide.

1.12 Inheritance

Inheritance lets us create a new class (a **derived** or **child** class) from an existing one (a **base** or **parent** class). The child **inherits** the parent's data and behaviour, so we don't write the same code over and over: Don't Repeat Yourself!

Example: lecturers, technicians, teaching-office staff and demonstrators are all **employees**, each with a name, an age, a salary…

- Put everything they have in common into an `Employee` class.
- Each kind of employee inherits it, and only adds what's special about it.

1.13 Polymorphism

Polymorphism lets us treat an object as its parent type, while it still behaves according to its own specific type.

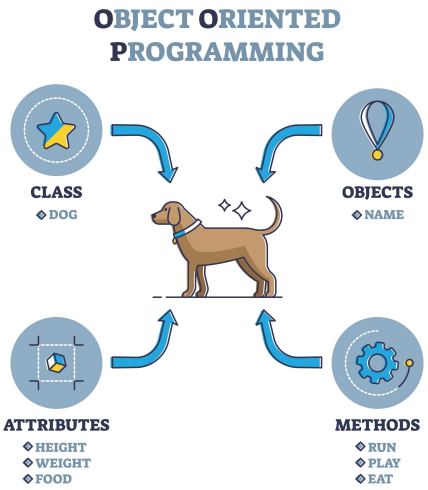
Example: keep a list of different shapes, tell every one of them to `draw()`, and each one draws itself correctly: circles as circles, squares as squares.

We'll understand this pillar properly when we get there, in Chapter 15.

2 Classes and objects

2.1 Classes and objects

- A **class** is a type that **you** define, a *program-defined* (or *user-defined*) type. Think of it as a **blueprint**.
- An **object** is one particular thing built from that blueprint: an **instance** of the class.



`int` is a type and `int x{5};` is an object of that type. In the same way, `Date` will be a class, and `Date today{2026, 2, 26};` an object.

2.2 Defining a class

A class is defined just like a struct, with the keyword `class` instead of `struct`:

```
class Employee {
    int m_id{};   // member variables (also called data members)
    int m_age{};  // m_ stands for "member"
    double m_wage{};
}; // don't forget this semicolon!
```

The `m_` prefix marks **member variables**. You'll see it a lot in other people's code. Is it a good idea? That's the next slide.

2.3 Naming member variables

- **`m_` prefix**, `m_age`: used by many companies. Handy in your IDE: type `m_` and it lists all the members.
- **Trailing underscore**, `age_`: what Google's C++ style guide recommends.
- **No decoration**, `age`: used by many modern open-source libraries.

Class Data Members

Data members of classes, both static and non-static, are named like ordinary nonmember variables, but with a trailing underscore. The exception to this is static constant class members, which should follow the rules for naming constants.

```
class TableInfo {
 public:
  ...
  static const int kTableVersion = 3;  // OK - constant naming.
  ...

 private:
  std::string table_name_;             // OK - underscore at end.
  static Pool<TableInfo>* pool_;       // OK.
};
```

These conventions exist to avoid **name shadowing** in constructors (Chapter 11). With modern C++ we won't need them, so in this course I'll mostly use plain names. I may still slip in an `m_` now and then, on purpose or out of habit!

2.4 `struct` versus `class`

Technically, `struct` and `class` are almost identical. The **only** differences are the defaults:

	`struct`	`class`
members are…	`public` by default	`private` by default
inheritance is…	`public` by default	`private` by default

Anything you can write with one, you can write with the other. In practice, we use them for different jobs:

- Use a `struct` for **passive data**: a bundle of values with no rules to enforce.
- Everything else is a `class`.

2.5 `struct` versus `class`: Google's C++ style

Structs vs. Classes

> Use a `struct` only for passive objects that carry data; everything else is a `class`.
>
> The `struct` and `class` keywords behave almost identically in C++. We add our own semantic meanings to each keyword, so you should use the appropriate keyword for the data-type you're defining.
>
> `structs` should be used for passive objects that carry data, and may have associated constants. All fields must be public. The struct type itself must not have invariants that imply relationships between different fields, since direct user access to those fields may break those invariants, but users of a struct may have requirements and guarantees on particular uses of it. Constructors, destructors, and helper methods may be present; however, these methods must not require or enforce any invariants.
>
> If more functionality or invariants are required, or struct has wide visibility and expected to evolve, then a `class` is more appropriate. If in doubt, make it a `class`.
>
> For consistency with STL, you can use `struct` instead of `class` for stateless types, such as traits, template metafunctions, and some functors.
>
> Note that member variables in structs and classes have different naming rules.

The keywords behave almost identically; *we* give each one its meaning, so pick the one that says what you mean.

2.6 A class can behave like a struct

Listing 1 `class_just_like_struct.cpp`

```cpp
#include <iostream>

class Date { // we changed struct to class...
  public:    // ...and added this access specifier
    int day{};
    int month{};
    int year{};
};

void print_date(const Date& date) {
    std::cout << date.day << '/' << date.month << '/' << date.year << '\n';
}

int main() {
    Date date{26, 2, 2026};
    print_date(date);
    return 0;
}
```

```
26/2/2026
```

Make all the members `public`, and a `class` behaves just like a `struct`.

2.7 You've been using classes all along

`std::string`, `std::string_view` and `std::vector` are all classes. In fact, most types in the standard library are.

> **!** Key idea
>
> Classes are the heart and soul of C++. They're so fundamental that C++ was originally called "C with Classes"!

3 Member functions

3.1 A non-member function

A class type can contain functions as well as data. Functions that belong to a class are **member functions**; all others are **non-member functions**. (I'll use `struct`s here, to show that all of this applies to them too.)

Listing 2 `non_member_function_example.cpp`

```cpp
#include <iostream>

struct Date {
    int year{}, month{}, day{};
};

void print(const Date& date) { // a non-member function
    std::cout << date.year << '/' << date.month << '/' << date.day << '\n';
}

int main() {
    Date today{2026, 2, 26};
    today.day = 27; // select a member with .
    print(today);   // pass the object as an argument
    return 0;
}
```

```
2026/2/27
```

3.2 A member function

Listing 3 `member_function_example.cpp`

```cpp
#include <iostream>

struct Date {
    int year{}, month{}, day{};

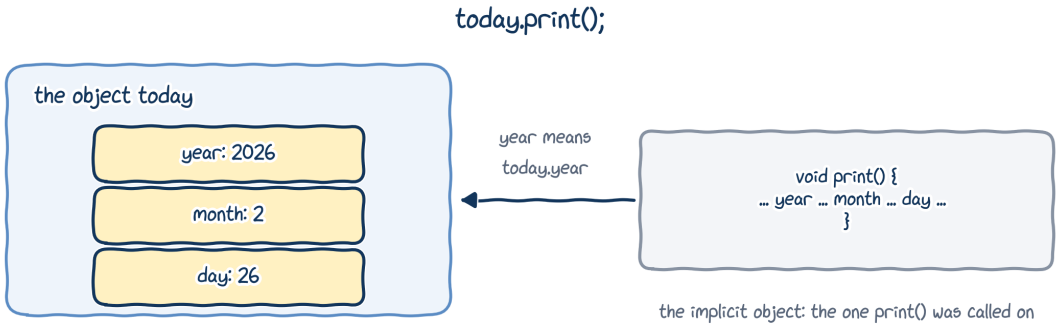
    void print() { // a member function
        std::cout << year << '/' << month << '/' << day << '\n';
    }
};

int main() {
    Date today{2026, 2, 26};
    today.day = 27; // select a member variable with .
    today.print();  // ...and a member function too
    return 0;
}
```

```
2026/2/27
```

`print()` is now **inside** `Date`, and we call it with `today.print()`.

3.3 The implicit object



- With a non-member function we write `print(today)`; with a member function, `today.print()`. We don't pass `today` as an argument.
- Instead, `today` is passed **implicitly**. It's called the **implicit object**: the object the member function was called on.
- Inside `print()`, a bare member name like `year` means *the implicit object's* `year`: `today.year`.

3.4 Coming from Python: where is `self`?

Listing 4 `date_class.py`

```python
class Date:
    def __init__(self, year=2026, month=2, day=28):
        self.year = year
        self.month = month
        self.day = day

    def print(self):
        print(f"{self.year}/{self.month}/{self.day}")


today = Date(2026, 2, 26)
today.print()      # the usual way...
Date.print(today)  # ...and what it really means: today is passed as self
```

```
2026/2/26
2026/2/26
```

Python passes the implicit object explicitly, as `self`: `Date.print(today)` is what `today.print()` really means. C++ has no `self` parameter; its version of `self`, called `this`, comes in Chapter 11.

3.5 Python checks late...

Listing 5 `date_class_wrong_object.py`

```python
class Date:
    def __init__(self, year, month, day):
        self.year, self.month, self.day = year, month, day

    def print(self):
        print(f"{self.year}/{self.month}/{self.day}")


Date.print("khiem")  # Python only complains when the line runs
```

```
AttributeError: 'str' object has no attribute 'year'
```

Python only discovers the problem when that line **runs**: maybe during your demo, maybe in front of a client.

3.6 ...or never

Listing 6 `date_class_nonsense.py`

```python
class Date:
    def __init__(self, year, month, day):
        self.year, self.month, self.day = year, month, day

    def print(self):
        print(f"{self.year}/{self.month}/{self.day}")


class Foo:
    def __init__(self):
        self.year, self.month, self.day = "one", "two", 3


Date.print(Foo())  # runs without complaint... and prints nonsense
```

```
one/two/3
```

This runs without a single complaint, and prints nonsense. In C++, `Date::print` can only be called on a `Date`, and the compiler checks it before the program ever runs.

3.7 Member functions can be overloaded

Listing 7 `overload_member_functions.cpp`

```cpp
#include <iostream>
#include <string>

struct Date {
    int year{}, month{}, day{};
    void print() { std::cout << year << '/' << month << '/' << day << '\n'; }
    void print(const std::string& prefix) { // an overload
        std::cout << prefix << year << '/' << month << '/' << day << '\n';
    }
};

int main() {
    Date today{2026, 2, 26};
    today.print();                // calls Date::print()
    today.print("The date is: "); // calls Date::print(const std::string&)
    return 0;
}
```

```
2026/2/26
The date is: 2026/2/26
```

Just like non-member functions (Chapter 05), member functions can be overloaded.

3.8 Structs and member functions: a bit of history

- In C, structs could only hold data, never functions.
- When designing C++, Bjarne Stroustrup had to decide whether structs, inherited from C, should be allowed member functions. He decided they should.
- So in modern C++, it's perfectly fine for a struct to have member functions.

Structs and classes are both called **class types**.

4 `const` objects and `const` member functions

4.1 `const` objects

A `const` variable must be initialized, and can't be changed (Chapter 04). Objects of class type can be `const` too, and then their members can't be changed either:

Listing 8 `modify_members_of_const_objects.cpp`

```cpp
struct Date {
    int year{}, month{}, day{};
    void increment_day() { ++day; }
};

int main() {
    const Date today{2026, 2, 26}; // a const object

    today.day += 1;         // error: can't modify a member of a const object
    today.increment_day(); // error: this function would modify it too
    return 0;
}
```

```
error: assignment of member 'Date::day' in read-only object
error: passing 'const Date' as 'this' argument discards qualifiers
```

4.2 `const` objects can't call non-`const` member functions

This one may surprise you. `print()` only *reads* the date:

Listing 9 `const_objects_call_non_const_mem_functions.cpp`

```cpp
#include <iostream>

struct Date {
    int year{}, month{}, day{};
    void print() { // not const
        std::cout << year << '/' << month << '/' << day << '\n';
    }
};

int main() {
    const Date today{2026, 2, 26};
    today.print(); // error: print() might modify today
    return 0;
}
```

```
error: passing 'const Date' as 'this' argument discards qualifiers
```

The compiler doesn't look inside `print()` to see what it does. As far as it knows, `print()` *might* change `today`, so it refuses.

4.3 `const` member functions

A **`const` member function** promises not to change the object, and not to call any non-`const` member function.

Listing 10 `const_member_functions.cpp`

```cpp
#include <iostream>

struct Date {
    int year{}, month{}, day{};

    void print() const { // a const member function
        std::cout << year << '/' << month << '/' << day << '\n';
        int next_year{year + 1}; // locals can still change
        ++next_year;
    }
};

int main() {
    const Date today{2026, 2, 26};
    today.print(); // fine
    return 0;
}
```

```
2026/2/26
```

Put `const` after the parameter list. Local variables can still change; only the object's members are protected.

4.4 A `const` member function can't change members

No surprise: break the promise, and the compiler stops you.

Listing 11 `const_member_functions_change_data_members.cpp`

```
struct Date {
    int year{}, month{}, day{};
    void increment_day(int days) const { // made const...
        day += days; // error: ...so it can't change a member
    }
};

int main() {
    Date today{2026, 2, 26};
    today.increment_day(4);
    return 0;
}
```

```
error: assignment of member 'Date::day' in read-only object
```

4.5 `const` member functions work on any object

Listing 12 `nonconst_objects_call_const_mem_functions.cpp`

```
#include <iostream>

struct Date {
    int year{}, month{}, day{};

    void print() const {
        std::cout << year << '/' << month << '/' << day << '\n';
    }
};

int main() {
    Date today{2026, 2, 26}; // not const
    today.print();           // fine
    return 0;
}
```

```
2026/2/26
```

Also no surprise: a function that promises *not* to change an object is safe on any object.

> **Best practice**
>
> A member function that doesn't (and never will) change the object should be `const`, so that it works on both `const` and non-`const` objects.

4.6 Wait a minute: what a mess!

How am I supposed to remember all this `const` stuff? I'm not Tony Stark or Spider-Man.

So I asked Gemini: *"There are too many things to remember about making functions and references const. Can you give me one or two golden rules?"* Here's its answer, shortened a bit:

> ! Two golden rules (covering 95% of cases)
>
> 1. **The read-only rule, for member functions:** if a function doesn't change the object's data, mark it `const`. It's a promise to the compiler.
> 2. **The efficient-pass rule, for parameters:** pass objects by `const&` (const reference), unless they're tiny or the function needs to change them.

Let's see both rules working together.

4.7 `const` objects via pass by `const` reference

- Passing a class-type argument **by value** copies it, which is slow. Passing **by reference** avoids the copy (Chapter 07).
- With a `const` reference, the function can't modify the argument, which is usually what we want.
- So besides `const` variables, there's a second common way to meet a `const` object: through a `const` reference parameter. **Inside that function**, the object is `const`.

4.8 Spot the problem

What's wrong with this code?

Listing 13 `const_objects_via_pass_by_const_ref_01.cpp`

```cpp
#include <iostream>

struct Date {
    int year{}, month{}, day{};

    void print() { // not const
        std::cout << year << '/' << month << '/' << day << '\n';
    }
};

void do_something(const Date& date) {
    date.print(); // error: date is const in here
}

int main() {
    Date today{2026, 2, 26}; // not const
    do_something(today);
    return 0;
}
```

```
error: passing 'const Date' as 'this' argument discards qualifiers
```

Inside `do_something()`, `date` is `const`, and `print()` isn't a `const` member function.

4.9 The fix is one word

Listing 14 `const_objects_via_pass_by_const_ref_02.cpp`

```cpp
#include <iostream>

struct Date {
    int year{}, month{}, day{};

    void print() const { // now const: the one-word fix
        std::cout << year << '/' << month << '/' << day << '\n';
    }
};

void do_something(const Date& date) {
    date.print(); // fine
}

int main() {
    Date today{2026, 2, 26};
    do_something(today);
    return 0;
}
```

```
2026/2/26
```

Make `print() const`, and both golden rules are satisfied.

5 Wrapping up

5.1 Summary

- OOP bundles data and behaviour into **classes**. Its four pillars are encapsulation, abstraction, inheritance and polymorphism.
- A **class** is a blueprint (a type); an **object** is an instance of it.
- `struct` and `class` differ only in their defaults: `public` versus `private`. Use `struct` for passive data, and `class` for everything else.
- **Member functions** belong to a class and work on the **implicit object**, the one they're called on. C++ has no `self` parameter.
- A **`const` object** can only call **`const` member functions**. Make every member function that doesn't change the object `const`.
- Pass class objects by `const` reference.

5.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Introduction to object-oriented programming
2. Introduction to classes
3. Member functions
4. Const class objects and const member functions

Next up, Chapter 11: access specifiers and constructors.